

Code-Layout, Readability and Reusability

Date	26 June 2026
Team ID	SWTID-2026-9014
Project Name	AdForge AI: AI-Powered Advertisement Copy Generator
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Well-structured code is essential for maintainability and collaboration. Document the standards, patterns, and principles your team followed to ensure the codebase is clean, readable, and highly reusable. AdForge AI was developed with a strong emphasis on clean code principles, maintainability, and reusability. Every file in the project follows consistent formatting standards, logical organisation, and modular design patterns to ensure that any developer can understand, extend, or debug the codebase with minimal effort.

Code Quality & Structure Details

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	Folder/Directory Structure	<i>The project follows a clean MVC-inspired architecture separating concerns into distinct layers. The backend logic, routing, and AI integration are all contained in a single modular app.py file following Flask conventions. HTML templates are isolated in the templates/ folder using Jinja2. Static assets are separated into static/css/ for styles and static/js/ for JavaScript. Environment configuration is kept outside the codebase in a .env file. This separation ensures that frontend, backend, and configuration concerns never</i>	<i>app.py — Backend logic and routes · templates/index.html · templates/about.html · templates/how_it_works.html · static/css/style.css · static/js/script.js · .env · .env.example · requirements.txt · test_app.py · ngrok_start.py</i>

		<i>overlap and each layer can be modified independently without affecting others.</i>	
2	Naming Conventions	Throughout the entire codebase consistent and descriptive naming conventions were followed. Python functions and variables use snake_case as per PEP 8 standards — for example build_ad_prompt(), is_unsafe_input(), validate_inputs(), and call_groq(). Constants are written in UPPER_SNAKE_CASE such as BLOCKED_TERMS, AD_REQUIRED_FIELDS, and TIPS_REQUIRED_FIELDS. HTML element IDs and JavaScript variables use camelCase such as adOutput, submitRefineForm, and loaderMessages. CSS class names use kebab-case such as .output-area, .platform-check, .btn-primary, and .loader-wrap. Flask route names match their functionality — /api/generate, /api/refine, /api/tips.	<i>app.py — snake_case functions and UPPER_SNAKE_CASE constants · static/js/script.js — camelCase variables and functions · static/css/style.css — kebab-case class names · templates/index.html — descriptive element IDs</i>
3	Code Readability & Comments	<i>Every Python function in app.py contains a descriptive docstring explaining its purpose, parameters, and return value. Inline comments are placed above every non-obvious logic block to explain why the code exists, not just what it does. Section dividers using dashes separate logical blocks such as Configuration, Safety, Prompt Builders, Groq API Service, Input Validation, and Routes —</i>	<i>app.py — docstrings on all 8 functions, section dividers, inline comments · static/js/script.js — function-level comments on all 12 functions · static/css/style.css — design system header comment, section comments · test_app.py — 6 labelled sections with 60 named tests</i>

		<i>making navigation effortless. In script.js every function has a single-line comment describing its role. In style.css a header comment block documents the design system including colour palette, typography choices, and design signature. In test_app.py every test section has a clearly labelled header and each individual test has a descriptive name string.</i>	
4	Modularity & Reusability	<i>The backend strictly follows the DRY (Do Not Repeat Yourself) principle by breaking all logic into single-purpose reusable functions. The sanitize() function is called on every user input field — reused across all three API routes. The is_unsafe_input() function is reused in all three endpoints. The validate_inputs() function accepts a fields parameter making it reusable for both the Ad Generator and Performance Tips forms with different field requirements. The three prompt builder functions build_ad_prompt(), build_refine_prompt(), and build_tips_prompt() are each independently callable. The call_groq() function is a single centralised API caller reused by all three routes. On the frontend the showOutput(), setLoading(), showAlert(), copyOutput(), and downloadOutput() functions are reused across all three panels without duplication. CSS custom</i>	<i>app.py — sanitize() reused 17 times · validate_inputs() reused across 3 routes · call_groq() reused by all 3 endpoints · static/js/script.js — showOutput(), setLoading(), showAlert() reused across 3 panels · static/css/style.css — CSS variables in :root reused across entire stylesheet</i>

		<i>properties defined in :root are reused throughout the entire stylesheet.</i>	
--	--	---	--

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
5	Formatting & Linting Tools	<i>Python code in app.py and test_app.py follows PEP 8 formatting standards throughout — consistent 4-space indentation, maximum line length of 79 characters where possible, blank lines between functions, and logical grouping of imports. Type hints are used on all Python function signatures for clarity — for example def sanitize(value: str, max_len: int = 500) -> str. CSS uses consistent 2-space indentation with each property on its own line and related rules grouped together with blank line separators. JavaScript uses consistent 2-space indentation, single quotes for strings, and semicolons at the end of every statement. All HTML templates use 2-space indentation with semantic elements and ARIA attributes for accessibility.</i>	<i>app.py — PEP 8 compliance, type hints on all functions · static/css/style.css — consistent 2-space indentation, grouped rules · static/js/script.js — consistent formatting, semicolons · templates/index.html — semantic HTML5 elements, 2-space indentation</i>
6	Error Handling Patterns	<i>A consistent and centralised error handling pattern is applied across all three Flask API routes. Every route is wrapped in a try/except block that catches all exceptions from the Groq API call and returns a structured JSON error response with a meaningful message. HTTP status codes are used semantically — 400 for missing required fields, 422 for unsafe or unprocessable content detected by the safety filter, and 502 for upstream Groq API failures. All errors are logged server-side using app.logger.error() for debugging. On the frontend every fetch() call is wrapped in a try/catch block that catches network errors and displays a user-friendly alert message.</i>	<i>app.py — try/except in generate_ads(), refine_ad(), performance_tips() · app.logger.error() on all API failures · 400 / 422 / 502 HTTP status codes · static/js/script.js — try/catch in submitAdForm(), submitRefineForm(), submitTipsForm() · showAlert() unified error display</i>

		<i>The showAlert() function provides a unified error display mechanism reused across all three panels. Errors never expose internal server details to the user.</i>	
--	--	---	--